Taoyu Lyu

(838) 934-3852 Taoyu9536@gmail.com github.com/ChrisLvvvv linkedin.com/in/taoyulyu

Education

- **University of Pennsylvania** **Philadelphia, PA**
  M.S.E. in Electrical Engineering *August 2026 – May 2028*
  - Intended focus: Computer Engineering
- **Rensselaer Polytechnic Institute** **Troy, NY**
  B.S. in Electrical Engineering, Minor in Economics *August 2022 – May 2026*
  - Dean's Honor List, final four graded terms
  - Selected coursework: Computer Architecture & Networks, Computer Hardware Design, Advanced Computer Hardware Design, Advanced Computer Systems, Embedded Control, Microelectronics Technology

Research Experience

Enhancing KV Cache Efficiency with Compression and Bit-Plane Eviction in Transformer Models **Aug. 2024 – Dec. 2024**
Undergraduate Research Program, Rensselaer Polytechnic Institute *Advisor: Prof. Tong Zhang; Mentor: Dr. Rui Xie*

- Profiled KV-cache memory growth in LLaMA 3.1 8B inference, characterizing FP16 cache footprint at approximately 128 KB per token and 1 GB for an 8K-token context.
- Designed a memory-budgeted KV-cache retention policy combining semantic clustering with temporal recency to prioritize important context tokens under constrained cache capacity.
- Prototyped token-to-cluster mapping and lookup mechanisms to support efficient assignment and retrieval of retained KV-cache entries during autoregressive decoding.
- Evaluated memory-quality tradeoffs by comparing semantic-temporal retention against recency-based retention and bit-plane precision-reduction baselines.

GPU Memory Management for High-Throughput LLM Serving **Aug. 2025 – Jan. 2026**
Undergraduate Research Program, Rensselaer Polytechnic Institute *Advisor: Prof. Liu Liu*

- Investigated GPU memory fragmentation in vLLM-style LLM serving, focusing on how PagedAttention maps logical KV-cache blocks to non-contiguous physical GPU memory during large-batch autoregressive inference.
- Developed PyTorch-based profiling scripts to sweep sequence length, request concurrency, and KV block size across synthetic serving workloads, targeting 128-2048 token contexts and 1-64 concurrent requests.
- Benchmarked contiguous over-provisioning against block-based KV-cache allocation to quantify internal fragmentation, reserved-token waste, and effective GPU memory utilization under variable-length prompts and outputs.
- Analyzed vLLM's request scheduling path across waiting, running, and swapped queues, identifying how CPU/GPU block allocation, preemption, and block reuse affect throughput under GPU memory pressure.

Professional Experience

Huichuang Medical Equipment **Beijing, China**
Research and Development Intern *Feb. 2025 – Aug. 2025*

- Designed and re-engineered schematics and multi-layer PCB layouts for an acquired transcranial magnetic stimulation (TMS) device platform, converting KiCad-based board files into Altium and rebuilding control, power, and cooling-module hardware designs.
- Proposed a hardware architecture update that consolidated discrete components into an integrated I2C-controlled IC, reducing routing complexity and lowering BOM cost.
- Authored a 30,000-word Chinese technical manual covering theoretical circuit analysis, calibration protocols, operation procedures, and QA workflows to support product transfer and internal standardization.

Yuwell Global **Danyang, Jiangsu, China**
Hardware Engineer Intern *May 2023 – Aug. 2023*

- Executed prototype bring-up for IR thermometers using Heimann sensor modules, performing SMT soldering and validating ADC signal paths with oscilloscope-based measurements.
- Diagnosed intermittent power-management faults in returned Japanese-market units and documented hardware-level findings to improve standby stability.

Test Engineer Intern *Apr. 2022 – Aug. 2022*

- Conducted EMI/EMS, ESD, and surge testing for medical devices under CISPR 22 and IEC 61000-4 procedures, supporting EU CE/FCC certification evidence and remediation reports for signal interference issues.

Projects

SRAM-Based 4-bit MAC Accelerator **Course Project**
ESE 5700 Digital Integrated Circuits; Cadence Virtuoso, Spectre, Pegasus, Quantus, Python *Fall 2026*

- Designed a transistor-level SRAM-fed MAC accelerator in 18 nm FinFET, integrating two custom 16×4 6T SRAM macros, a 4×4 unsigned multiplier, a 12-bit accumulator, and a pipelined sequencer; computes a 16-element dot product in 17 cycles at a 3.2 ns clock (54 ns latency, 19 pJ, 4.5×10^3 μm^2) at 0.8 V, TT.
- Sized the 6T bitcell through hand analysis and a cell-ratio sweep, then drew DRC/LVS-clean custom bitcell and mirrored array layouts with Quantus extraction; achieved post-layout hold/read SNM of 0.28/0.14 V and a 0.24 V write margin, with the cell remaining functional down to 0.5 V.
- Built the synchronous SRAM macro: registered inputs, a flat NAND4 row decoder chosen over a predecoded design via logical-effort comparison (22% lower core delay), precharge/equalization, tri-state write drivers, full-swing read path with output latch, and a 64-stage delay-chain controller generating precharge, wordline, write, and sense pulses; reached 1.2 ns read access and a 3.2 ns minimum period.
- Derived wordline pulse budgets from extracted bitline load and cell current (0.47 ns read, 0.18 ns write) and verified every control-edge ordering margin, including a passing run with 20% added bitline capacitance.
- Designed a transmission-gate master-slave register with a shared nonoverlapping clock generator; characterized clock-to-Q, polarity-dependent setup/hold, clock capacitance, and clock energy under both the 9-register SRAM and 21-register MAC clock fanouts.
- Implemented the datapath as a NAND2-based array multiplier and a 12-bit ripple-carry adder; calibrated a logical-effort model from FO1/FO4 simulations and upsized 22 carry-path gates, reducing adder delay by 8.5% with the predicted improvement within 7 ps of simulation.
- Designed the sequencer (one-hot FSM, 4-bit counter, and pipeline-control registers) to overlap SRAM reads with accumulation, with synchronous reset, mid-computation restart, external load muxing, and accumulator clear/load/hold implemented through data-input selection rather than clock gating.
- Closed system timing: the hand-estimated 2.0 ns SRAM-to-accumulator critical path matched the 2.1 ns measured arrival; a clock-period scan showed the limit was SRAM write-off-to-precharge ordering rather than the datapath (1.1 ns slack); hold was checked on all 39 DFFs, all with positive margin.
- Automated verification with Python-driven Spectre flows (stimulus generation, waveform grading, setup/hold boundary searches, period scans) across exhaustive multiplier tests, 384 datapath vectors, block-level unit tests, and full-system test vectors plus reset/restart sequences, all passing.
- Measured energy with separate supply probes per block: 1.2 pJ/MAC including memory and 0.45 pJ/MAC excluding it, with SRAM consuming 62%; estimated area with a bitcell-calibrated density method, showing periphery dominates at this array size.

Out-of-Order Processor and Memory System Design **Course Project**
ECSE 4780 Advanced Computer Hardware Design; SystemVerilog, ModelSim, Intel Quartus Prime Lite *Spring 2026*

- Implemented a modular SystemVerilog out-of-order processor datapath with two-wide dispatch, RAT-backed register renaming, an 8-entry reorder buffer, reservation stations, load-store queue, multi-cycle functional units, common data bus writeback, and backend resource backpressure.

- Designed precise commit and recovery logic supporting in-order retirement, completion tracking, exception recognition, trap visibility at the ROB head, younger-instruction flush across ROB/RS/LSQ/RAT state, late-result masking, and retirement-time store commit.
- Built and verified memory-system RTL including an 8 KiB direct-mapped write-back/write-allocate cache with FSM-based miss handling and a two-core MESI coherence model with private L1 caches, shared-bus transactions, L2 responses, and ModelSim testbenches covering dependency, scheduling, cache, coherence, and exception cases.

RISC-V Processor RTL Design and Pipeline Hazard Control **Course Project**
SystemVerilog, ModelSim 20.1, Intel Quartus Prime Lite, RV32I Datapath and Control *Fall 2025*

- Implemented RV32I processor RTL across single-cycle, multi-cycle, and five-stage pipelined datapaths, integrating instruction decode, immediate generation, ALU control, register-file access, PC update logic, load/store paths, and branch/jump handling.
- Designed FSM-based control and pipeline hazard logic, including fetch/decode/execute/memory/writeback pipeline registers, forwarding, load-use stalls, branch/jump flushes, x0-safe dependency handling, and ModelSim testbenches for register, memory, control-flow, stall, and flush correctness.

Embedded Robotic Control System on RPI-RSLK Platform **Course Project**
C, MSPM0G3507 LaunchPad, RPI-RSLK, GPIO, Timers, PWM, ADC, I2C, Encoders *Fall 2024*

- Developed embedded C control software for the RPI-RSLK robotic car on the MSPM0G3507 LaunchPad, integrating GPIO, timer interrupts, PWM motor control, ADC-based potentiometer inputs, I2C sensor communication, encoder feedback, and H-bridge motor drivers.
- Implemented and tuned motion-control routines for predetermined path following, remote speed/steering control, and gesture-based driving, using encoder-derived wheel measurements, differential motor control, heading estimation, and on-board hardware testing.

Technical Skills

Languages: C/C++, Python, CUDA C/C++ fundamentals, SystemVerilog, Verilog, MATLAB, Bash, Assembly (RISC-V, MIPS, x86), LaTeX
Systems & ML: Linux/WSL, Git, GDB, CMake, perf, PyTorch, Hugging Face Transformers, vLLM
Hardware & Embedded: STM32, MSPM0G3507 LaunchPad, Raspberry Pi, RPI-RSLK robotic platform, PCB design (Altium, KiCad), GPIO, I2C/SPI/UART, PWM, ADC, quadrature encoders, PID control, oscilloscope, DMM, logic analyzer, SMT soldering
EDA & Design Tools: Intel Quartus Prime Lite, ModelSim, Xilinx Vivado, LTspice, Code Composer Studio, Venus Simulator, Siemens NX